

## ✓ Top Interview Questions on Arrays

### 1. Reverse an Array

**Q: Write a program to reverse an array.**

Answer:

java

 Copy code

```
int arr[] = {1, 2, 3, 4, 5};  
for(int i = arr.length - 1; i >= 0; i--) {  
    System.out.print(arr[i] + " ");  
}
```

Explanation: We start from the last index and print till the first index. Time Complexity  $\rightarrow O(n)$ .

## 2. Find Maximum and Minimum

**Q: Find the largest and smallest element in an array.**

Answer:

java

 Copy code

```
int arr[] = {10, 20, 5, 30, 15};
int max = arr[0], min = arr[0];

for(int i=1; i<arr.length; i++) {
    if(arr[i] > max) max = arr[i];
    if(arr[i] < min) min = arr[i];
}

System.out.println("Max: " + max + ", Min: " + min);
```

**Explanation:** Compare each element with current max and min. Time Complexity → **O(n)**.

### 3. Search an Element

**Q:** Search if a number exists in an array.

**Answer:**

java

 Copy code

```
int arr[] = {10, 20, 30, 40};
int key = 30;
boolean found = false;

for(int i=0; i<arr.length; i++) {
    if(arr[i] == key) {
        found = true;
        break;
    }
}

System.out.println(found ? "Found" : "Not Found");
```

**Explanation:** Linear search checks every element. Time Complexity → **O(n)**.

#### 4. Find Missing Number (1 to n)

**Q:** One number is missing in an array containing numbers from 1 to n. Find it.

**Answer:**

java

 Copy code

```
int arr[] = {1, 2, 4, 5}; // missing 3
int n = arr.length + 1;  // total numbers should be 5
int sum = n*(n+1)/2;

for(int num : arr) {
    sum -= num;
}

System.out.println("Missing number: " + sum);
```

**Explanation:** Use sum of natural numbers formula. Time Complexity  $\rightarrow O(n)$ .

## 5. Find Duplicates

**Q:** Find duplicate elements in an array.

**Answer:**

java

 Copy code

```
int arr[] = {1, 2, 3, 2, 4, 5, 1};

for(int i=0; i<arr.length; i++) {
    for(int j=i+1; j<arr.length; j++) {
        if(arr[i] == arr[j]) {
            System.out.println("Duplicate: " + arr[i]);
        }
    }
}
```

## 6. Find Pair with Given Sum

**Q:** Find pairs of numbers whose sum is equal to a given number.

**Answer:**

```
java Copy code  
  
int arr[] = {2, 4, 3, 7, 5, 8};  
int target = 10;  
  
for(int i=0; i<arr.length; i++) {  
    for(int j=i+1; j<arr.length; j++) {  
        if(arr[i] + arr[j] == target) {  
            System.out.println(arr[i] + " + " + arr[j] + " = " + target);  
        }  
    }  
}
```

**Explanation:** Check all pairs. Time Complexity  $\rightarrow O(n^2)$ ,  
optimized with HashSet  $\rightarrow O(n)$ .

## 7. Rotate an Array

**Q:** Rotate the array to the right by k steps.

**Answer:**

java

 Copy code

```
int arr[] = {1,2,3,4,5};
int k = 2;
int n = arr.length;
int result[] = new int[n];

for(int i=0; i<n; i++) {
    result[(i+k)%n] = arr[i];
}

for(int num : result) {
    System.out.print(num + " ");
}
```

**Explanation:** Use modulo for shifting elements. Time Complexity  $\rightarrow O(n)$ .

## 8. Second Largest Element

**Q:** Find the second largest element in the array.

**Answer:**

java

 Copy code

```
int arr[] = {10, 20, 4, 45, 99};
int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;

for(int num : arr) {
    if(num > first) {
        second = first;
        first = num;
    } else if(num > second && num != first) {
        second = num;
    }
}

System.out.println("Second largest: " + second);
```

## 9. Sort an Array

**Q:** Sort array elements in ascending order.

**Answer:**

java

 Copy code

```
int arr[] = {5, 2, 8, 1, 3};
Arrays.sort(arr);

for(int num : arr) {
    System.out.print(num + " ");
}
```

**Explanation:** Arrays.sort() uses Dual-Pivot QuickSort for primitives. Time Complexity →  **$O(n \log n)$** .

## 10 Matrix Addition (2D Array):

Add two matrices

```
java Copy code  
  
int a[][] = {{1,2},{3,4}};  
int b[][] = {{5,6},{7,8}};  
int sum[][] = new int[2][2];  
  
for(int i=0; i<2; i++) {  
    for(int j=0; j<2; j++) {  
        sum[i][j] = a[i][j] + b[i][j];  
        System.out.print(sum[i][j] + " ");  
    }  
    System.out.println();  
}
```

**Explanation:** Add element by element. Time Complexity →  $O(m*n)$ .